Lab 1 — System Cal

Name: Mahdi Davoodzadeh
SN: 405725536
Repository: https://github.com/deep-observability-lab/OS-Tracing.git

1. Objective

In this lab, I completed two related experiments:

1. I used `strace` to inspect the system calls of a supplied C program. I compared tracing only its parent process with following the process created by `fork()`.
2. I used `stress-ng` to run four matrix-multiplication workers, recorded kernel scheduling and interrupt events with LTTng, and analyzed the resulting Common Trace Format (CTF) data in Trace Compass.

I aimed to connect source-level operations to actual system calls, identify worker processes and their CPU placement, distinguish runnable and sleeping states, and support my conclusions with trace counts, measured CPU time, and screenshots.

2. Experimental Environment

I performed the experiments on my Ubuntu laptop. I saved the lab files under:

```
/home/elor/Documents/Learning/MSC/Stack-Protocol/course-exercises/lab1/
```

I recorded the following configuration before or during the experiments:

Component	Version / configuration
Operating system	Ubuntu 24.04.5 LTS
Kernel used for **scheduler tracing**	`6.17.0-40-generic`
Logical CPUs	16
GCC	13.3.0
Python	3.12.3
`strace`	6.8
`stress-ng`	0.17.06
LTTng Tools	2.13.11
LTTng kernel modules	2.13.23 (built from source)
Trace Compass	12.1.0
Secure Boot	Disabled

The machine initially ran kernel `7.0.0-42-generic`. Although LTTng userspace tracing worked, `sudo lttng list -k` initially returned **“Kernel tracer not available.”** I therefore temporarily booted the already-installed `6.17.0-40-generic` kernel and built LTTng Modules 2.13.23 against its matching headers. The build and installation succeeded; I confirmed the kernel tracer and scheduler probe were loaded, and `sudo lttng list -k` then listed the required scheduler and IRQ tracepoints. I retained the build and installation logs for reproducibility.

I retained the newer kernel alongside kernel 6.17. All scheduler measurements reported below are from **kernel 6.17.0-40**.

3. Experiment A — System Call Tracing with `strace`

3.1. Program and preparation

I cloned the lab repository and copied `Lab 1/src/simple_program.c` into `lab1/src/`. I compiled the supplied program without modifying its source:

```
gcc -g -O0 -Wall -Wextra src/simple_program.c -o bin/simple_program
```

GCC reported one non-fatal warning: the variable `num` was declared but never used. The executable was a 64-bit, dynamically linked ELF binary with debug information.

The program opened `output.txt` for writing, wrote `Hello World!`, closed the file, then called `fork()`. The parent and child printed their process IDs on separate branches. I first ran it normally and verified that `output.txt` contained:

```
Hello World!
```

Both branches printed their messages, and the parent command returned exit code 0. The parent and child executed independently after `fork()`, with their output order determined by scheduling.

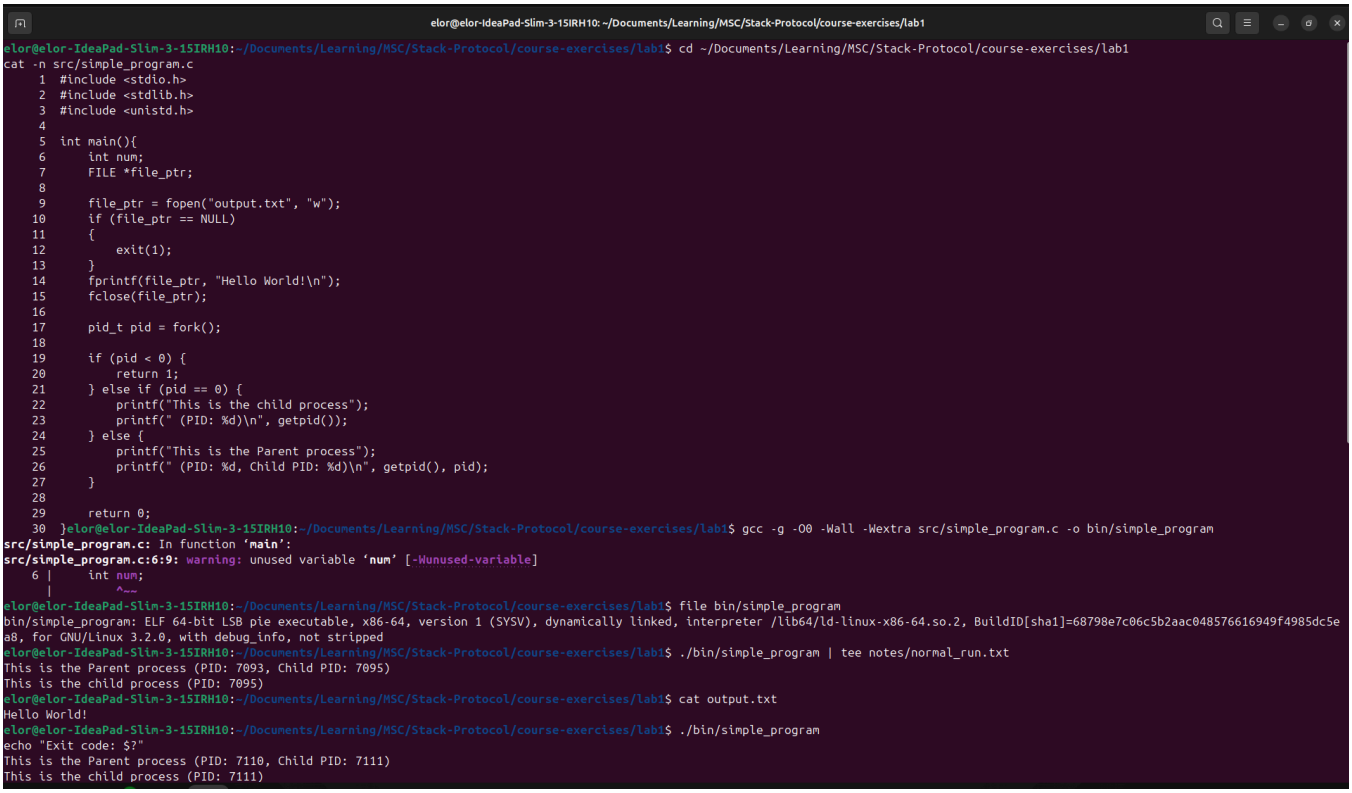


Figure 1 — Program preparation and normal execution. The terminal shows the supplied source, compilation with the unused-variable warning, the ELF executable information, parent/child output, and the contents of `output.txt`.

3.2. Trace collection

I traced the program once without following children and once with `-f`:

```
strace -tt -T -s 200 \
  -o logs/strace/without_f.log \
  ./bin/simple_program > logs/strace/without_f.stdout

strace -f -tt -T -s 200 \
  -o logs/strace/with_f.log \
  ./bin/simple_program > logs/strace/with_f.stdout
```

I used `-tt` for wall-clock timestamps, `-T` for per-call elapsed time, `-s 200` for longer string output, and `-o` to retain the detailed results. I then ran two separate profiling commands with `-c` and `-f -c` to obtain per-system-call summaries:

```
strace -c -o logs/strace/without_f_summary.txt \
  ./bin/simple_program > /dev/null

strace -f -c -o logs/strace/with_f_summary.txt \
  ./bin/simple_program > /dev/null
```

3.3. Results: tracing the parent versus following children

Measurement	Without `-f`	With `-f`
Lines in the detailed trace	42	56
Processes whose calls were traced	Parent only	Parent and child
Total calls reported by `strace -c`	41	46
`write` calls in the `-c` summary	2	3
`getpid` calls in the `-c` summary	1	2
`fstat` calls in the `-c` summary	4	5
Total call time reported by `-c`	0.000163 s	0.001120 s

I observed that the `-f` trace included events belonging to both the parent (PID **7378**) and the child (PID **7379**) in that particular run. Without `-f`, the detailed trace showed only its parent process (PID **7140** in a different run), even though the program still created a child.

Selected events from `with_f.log` illustrate the sequence:

```
7378 openat(..., "output.txt", O_WRONLY|O_CREAT|O_TRUNC, 0666) = 3
7378 write(3, "Hello World!\n", 13) = 13
7378 close(3) = 0
7378 clone(...) = 7379
7378 getpid(...) = 7378
7378 write(1, "This is the Parent process ...", 56) = 56
7379 getpid(...) = 7379
7379 write(1, "This is the child process ...", 38) = 38
```

The actual C code called `fork()`, but the trace recorded `clone(...)` with `SIGCHLD` among its flags. On this system, glibc implemented the process-creation operation using the `clone` system call. The parent and child then each called `getpid()` and wrote to standard output. The `-f` output interleaved some calls using `<unfinished ...>` and `<... resumed>` entries, which showed concurrent calls from different processes.

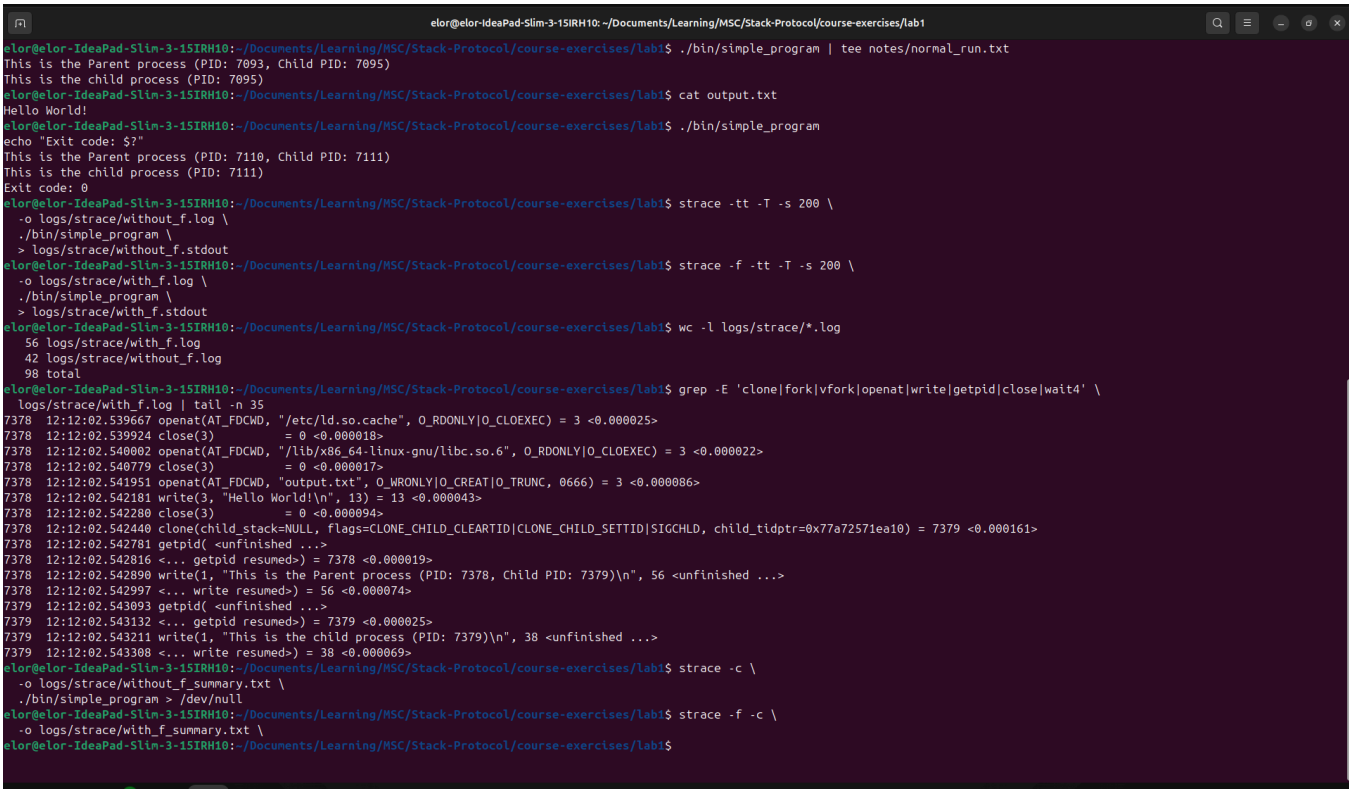


Figure 2 — Detailed `strace` comparison. The two tracing commands produce 42 lines without `-f` and 56 with `-f`. The extracted `with_f.log` records show `clone(...)` and the PID-prefixed calls from parent 7378 and child 7379.

3.4. Explanation of the observed system calls

I grouped the 21 distinct syscall names in the detailed traces by their purpose. Most memory-management and initialization calls occurred before the application reached its own file and process operations.

System call	What I observed / how I interpreted it
`execve`	Started the program image.
`access`	Checked for `/etc/ld.so.preload`; `ENOENT` indicated that the optional file was absent.
`openat`	Opened the dynamic-linker cache, the C library, and `output.txt`.
`read`	Read bytes from a file descriptor during loading.
`pread64`	Read data from a specified file offset, used during ELF/library loading.
`fstat`	Obtained descriptor metadata, including for files and standard output.
`close`	Closed descriptors; one close corresponded to `fclose(file_ptr)`.
`write`	Wrote `Hello World!` to FD 3 and process messages to FD 1.
`brk`	Managed the process heap boundary.
`mmap`	Created virtual-memory mappings for libraries and runtime data.
`mprotect`	Changed permissions of mapped memory regions.
`munmap`	Removed a mapping that was no longer needed.
`arch_prctl`	Set architecture-specific thread state, including the FS base on x86-64.
`set_tid_address`	Set up a thread-ID-related address for runtime/thread handling.
`set_robust_list`	Registered robust-futex information with the kernel.
`rseq`	Registered restartable-sequence support.
`prlimit64`	Queried resource limits, including the stack limit.
`getrandom`	Obtained random bytes used by the runtime.
`clone`	Created the child process requested by the C-level `fork()`.
`getpid`	Returned the current process ID for the parent or child message.
`exit_group`	Terminated a process with the corresponding exit status.

In addition to the 21 syscall names in the detailed traces, the separate `strace -c` profiling summaries reported `ioctl`, a descriptor-control system call. The program completed normally despite the failed `access` check for `/etc/ld.so.preload` and the failed `ioctl` calls in the profiling runs.

I also distinguished C library functions from kernel syscalls. For example, the C functions `fopen`, `fprintf`, and `fclose` ultimately led to `openat`, `write`, and `close`, but buffering and the runtime library determine the exact timing and sequence. Similarly, `printf` produced a `write` to standard output.

4. Experiment B — Kernel Scheduler Tracing with LTTng

4.1. Kernel tracer validation

Before collecting the main trace, I built LTTng Modules 2.13.23 for kernel `6.17.0-40-generic`. The build completed with exit code 0, the `.ko` files were installed, and `modinfo` identified matching module versions and kernel `vermagic` values.

I first recorded a small smoke test with only `sched_switch`, running `/bin/true` 100 times and sleeping for two seconds. Babeltrace read the resulting CTF files and reported 30,261 `sched_switch` events. This total captured scheduling activity across the whole machine, including the test processes and background tasks.

For reproducibility, I used the following commands from the downloaded `lttng-modules-2.13.23` source directory, after verifying that kernel 6.17 and its headers were installed:

```
make -j2 KERNELDIR="/lib/modules/$(uname -r)/build"
sudo make KERNELDIR="/lib/modules/$(uname -r)/build" modules_install
sudo depmod -a "$(uname -r)"
sudo lttng list -k
```

The build and installation logs were captured separately. The installation printed a warning about a missing `System.map` when performing its internal `depmod` step; I then ran `depmod` explicitly, and module discovery plus kernel event listing succeeded. These source-built modules were specific to the 6.17 kernel used in this experiment.

4.2. Main tracing configuration

I created a new session named `lab1-matrix4-v2` and explicitly enabled its kernel channel `schedchan`. I made this adjustment because my earlier trial session displayed a disabled default channel. In the working session, I confirmed `schedchan: [enabled]` before tracing.

The main event rules were:

```
sched_switch
sched_wak*                # includes sched_waking / sched_wakeup / sched_wakeup_new
sched_process_fork
sched_process_exit
irq_*
```

I attached the four requested kernel contexts: `vpid`, `vtid`, `procname`, and `prio`. I kept the channel in `discard` mode and checked its discarded-event counter after the capture.

Key configuration commands were:

```
sudo lttng create lab1-matrix4-v2 \
  --output="$PWD/logs/lttng/matrix4-v2"

sudo lttng enable-channel -k \
  --session lab1-matrix4-v2 schedchan

sudo lttng enable-event -k \
  --session lab1-matrix4-v2 \
  --channel schedchan \
  'sched_switch,sched_wak*,sched_process_fork,sched_process_exit,irq_*'

sudo lttng add-context -k \
  --session lab1-matrix4-v2 --channel schedchan \
  -t vpid -t vtid -t procname -t prio
```

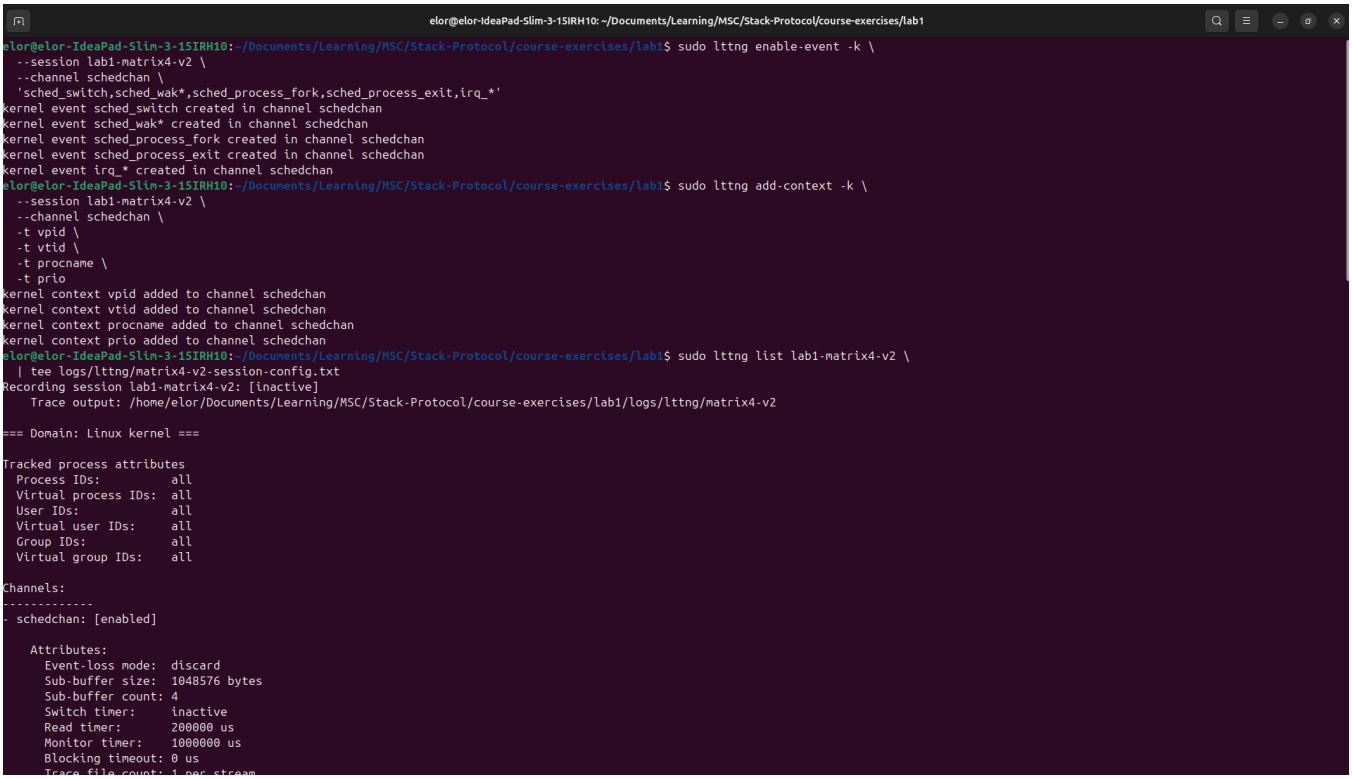


Figure 3 — LTTng event and context configuration. The terminal confirms creation of the requested event rules, addition of all four contexts, and an enabled `schedchan` channel.

4.3. Matrix-multiplication workload

I started tracing, executed the workload below, and then stopped tracing:

```
sudo lttng start lab1-matrix4-v2

stress-ng --matrix 4 --matrix-size 128 \
  --timeout 10s --metrics-brief

sudo lttng stop lab1-matrix4-v2
```

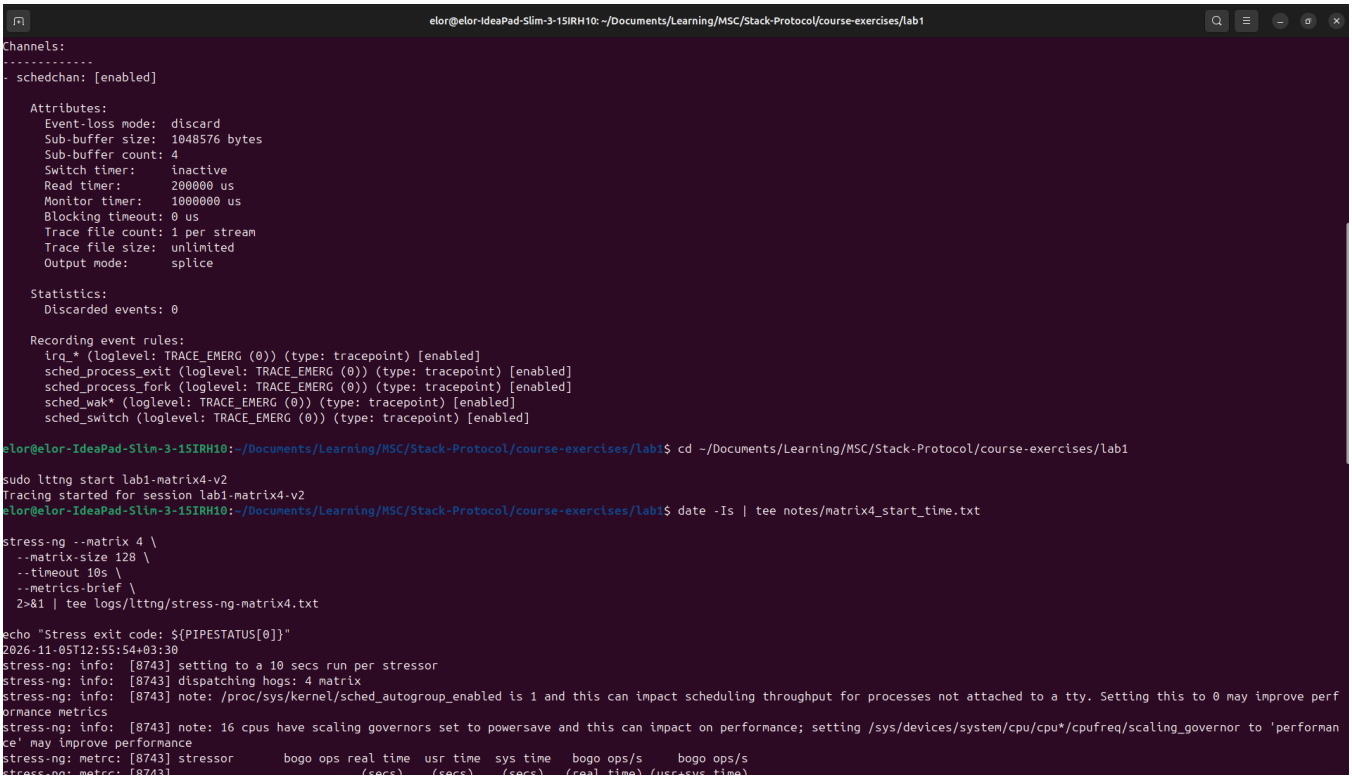


Figure 4 — Starting the tracing session and workload. The enabled channel and event rules are visible above the session start and the four-worker `stress-ng` command. The workload start timestamp is 12:55:54.

I chose four workers on a machine with 16 logical CPUs to generate sustained CPU demand while leaving other CPUs available for the operating system. The `stress-ng` run exited successfully and reported:

Workload metric	Observed value
Matrix workers	4
Matrix size	128 × 128
Workload duration	10.00 s
Total bogo operations	170,526
Bogo ops/s (real time)	17,051.11
Cumulative user CPU time	39.99 s
Cumulative system CPU time	0.00 s (rounded)
Workers passed / failed	4 / 0
Workload exit code	0
CTF trace directory size	14 MB
LTTng discarded events	0

I ran the experiment with `sched_autogroup_enabled = 1` and the `powersave` CPU scaling governor on all 16 logical CPUs, and recorded these settings as part of the experimental conditions.

I recorded the workload start at 12:55:54 and a post-workload stop-command timestamp at 12:56:15 (local time). Therefore, the CTF trace includes some background activity before and after the approximately 12:55:54–12:56:04 workload interval. I treated the complete trace and the workload-specific measurements as different scopes.

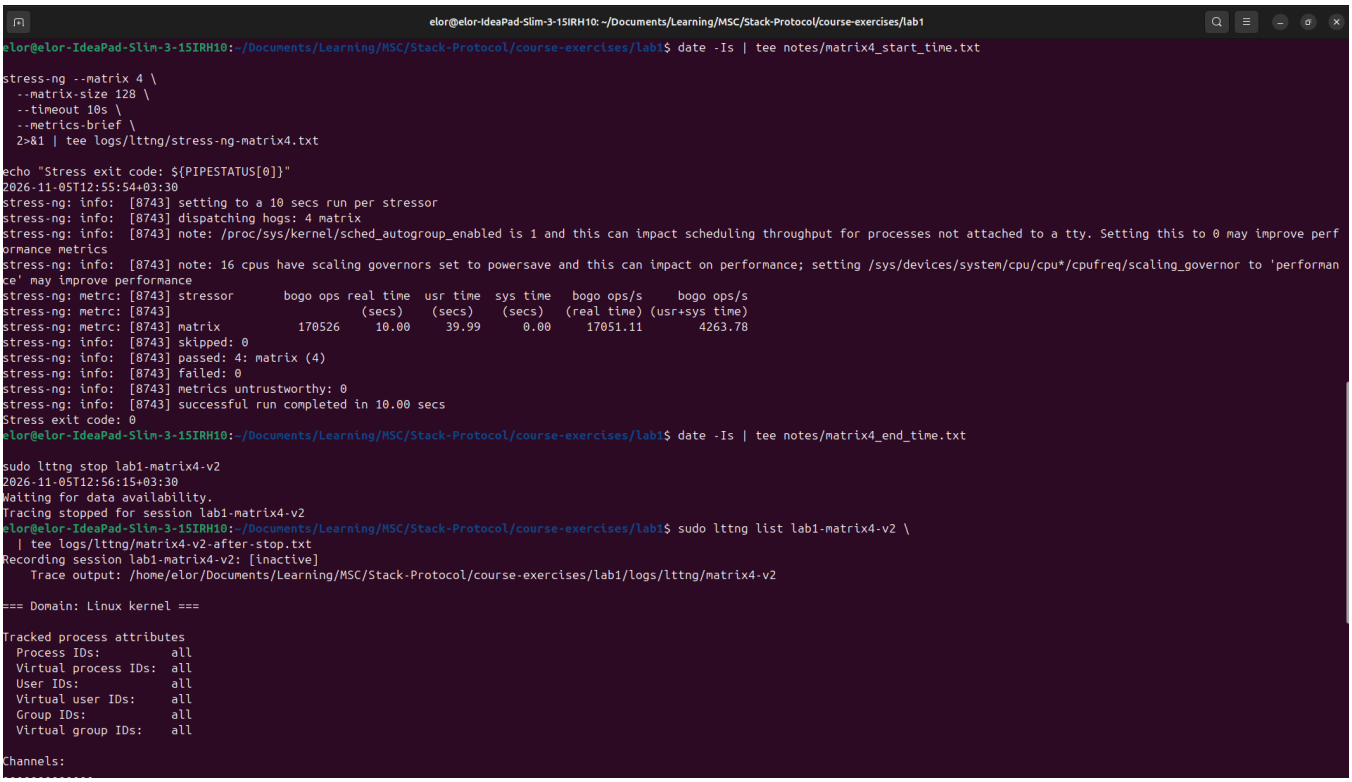


Figure 5 — Workload results and tracing stop. The terminal reports 170,526 bogo operations, a 10.00-second run, 39.99 seconds of cumulative user CPU time, and four passed workers. The later timestamp records the tracing stop command issued after the workload completed.

4.4. Trace integrity and event counts

I used Babeltrace to read the CTF data and counted events by name. It exited with code 0, its error output was empty, and the LTTng session reported 0 discarded events after stopping. I preserved the root-owned original trace and created a user-owned copy for Trace Compass; reading the copy returned the same 58,853 `sched_switch` events.

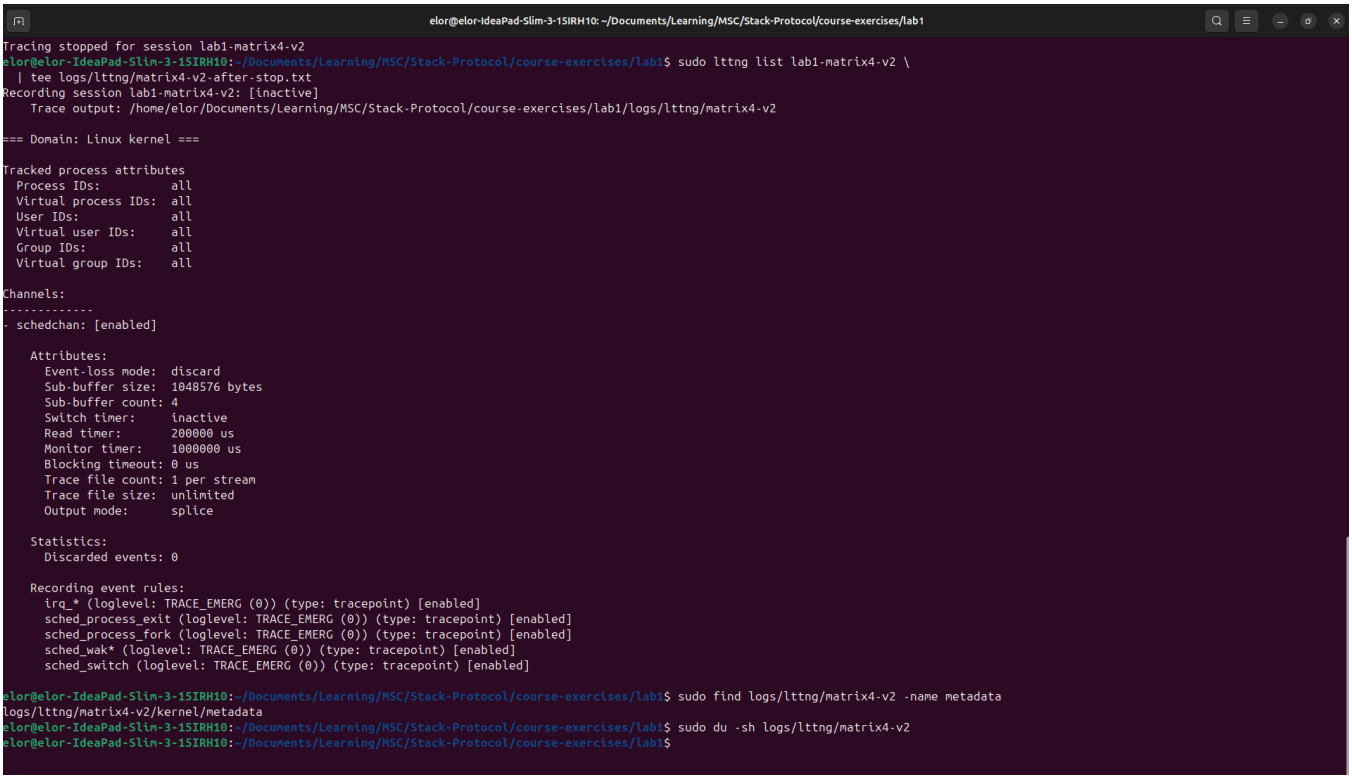


Figure 6 — Post-capture session verification. The session is inactive after stopping, `schedchan` reports 0 discarded events, and the CTF metadata file is located under `matrix4-v2/kernel/`.

Event name	Count in the full trace
`sched_switch`	58,853
`sched_waking`	29,736
`sched_wakeup`	29,736
`sched_wakeup_new`	29
`sched_process_fork`	29
`sched_process_exit`	29
`irq_handler_entry`	2,480
`irq_handler_exit`	2,480
`irq_softirq_raise`	46,441
`irq_softirq_entry`	46,412
`irq_softirq_exit`	46,412
Total	**262,637**

I verified the total count against the 262,637 events shown by Trace Compass Global Statistics. The selected-range statistics view reported `122,888` events within its selected interval. The global fork, wakeup, and IRQ totals describe system-wide activity, including `stress-ng` and background processes.

The `irq_handler_entry` and `irq_handler_exit` events mark the beginning and end of hardware interrupt handler execution. Soft IRQs perform deferred kernel processing: `irq_softirq_raise` records a request for that processing, while `irq_softirq_entry` and `irq_softirq_exit` mark the handler's execution boundaries. I observed interrupt activity alongside the matrix computation and counted the recorded event occurrences across the trace. These counts describe system-wide interrupt activity; the workers' CPU execution times are reported separately in Section 4.6.

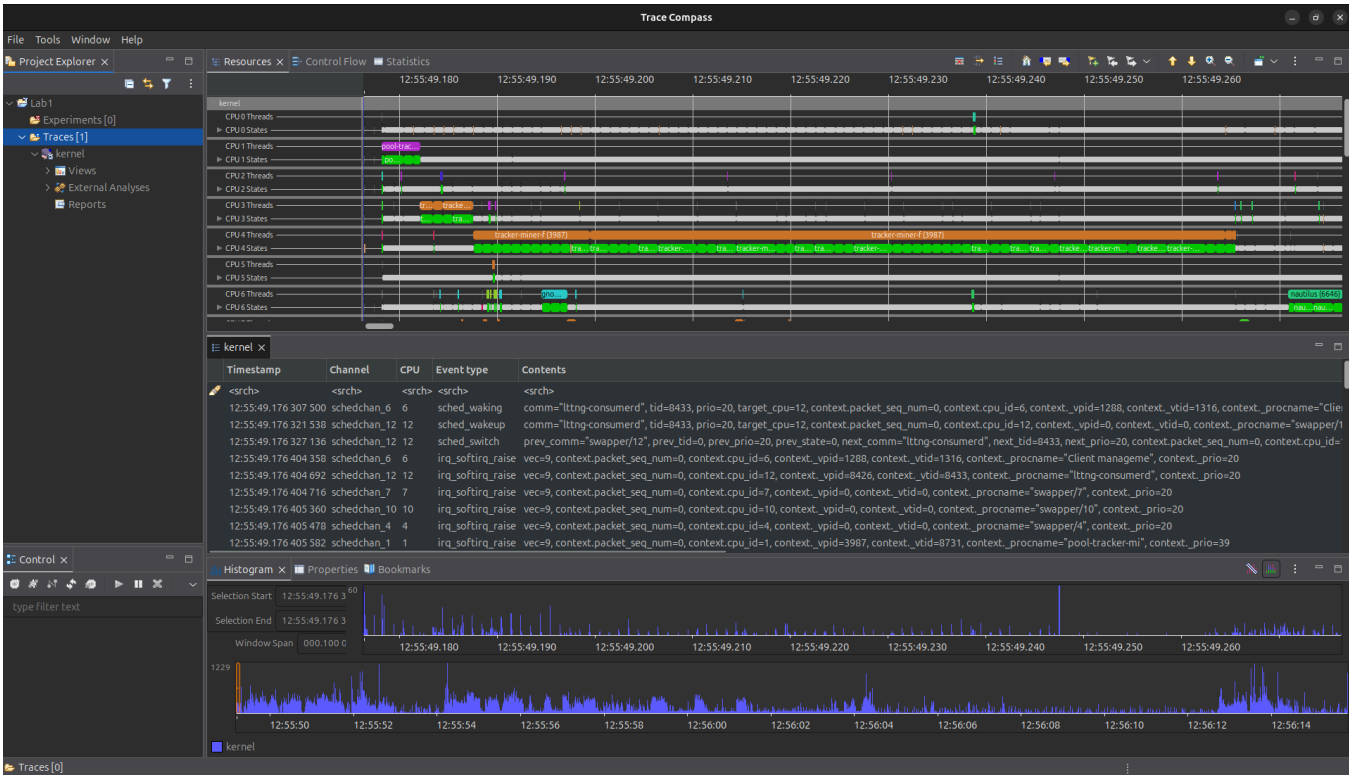


Figure 7 — Imported LTTng kernel trace and Events table. The `Lab1` project contains the kernel trace. The Events table displays scheduler and soft-IRQ events, while the Resources view and histograms show the surrounding activity.

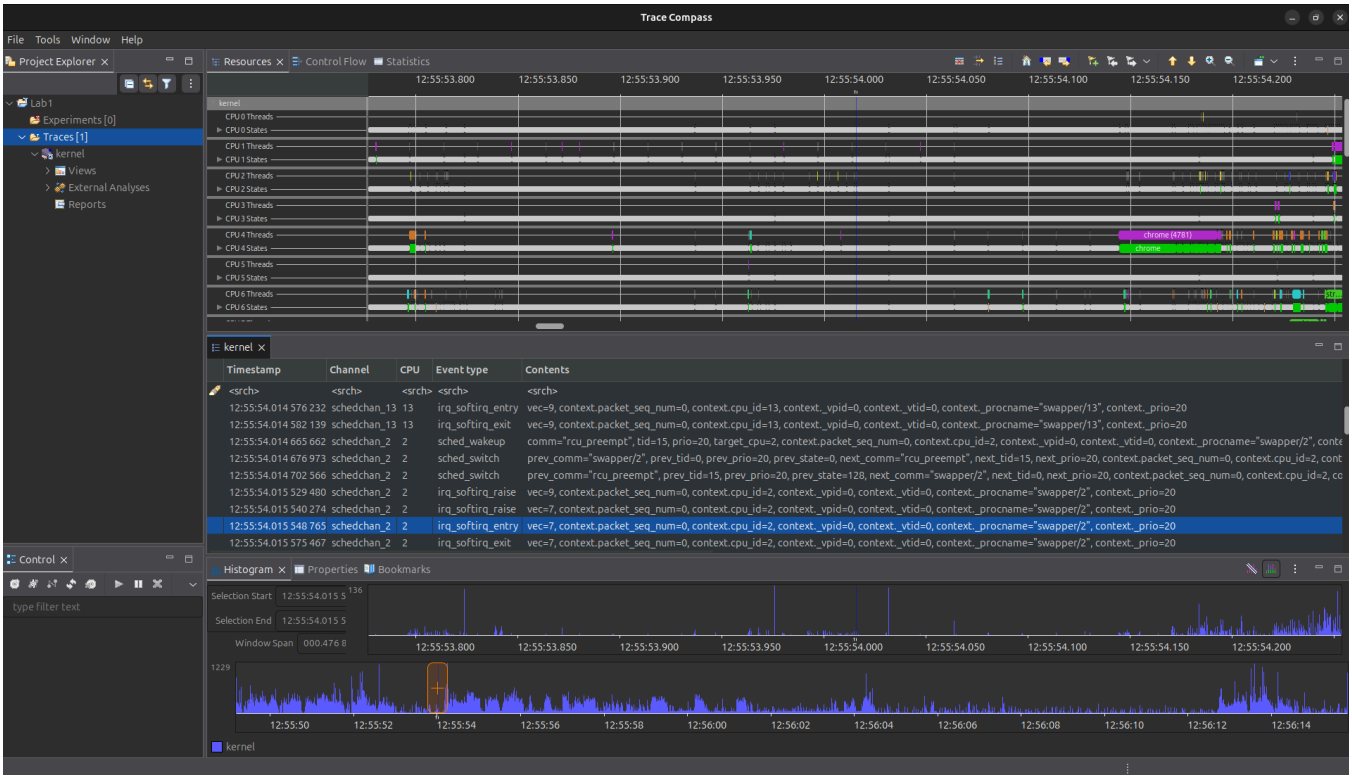


Figure 8 — Event inspection near workload startup. The selected `irq_softirq_entry` event occurs on CPU 2 at approximately 12:55:54.015, before the matrix workers begin execution. Its contents expose the recorded context fields.

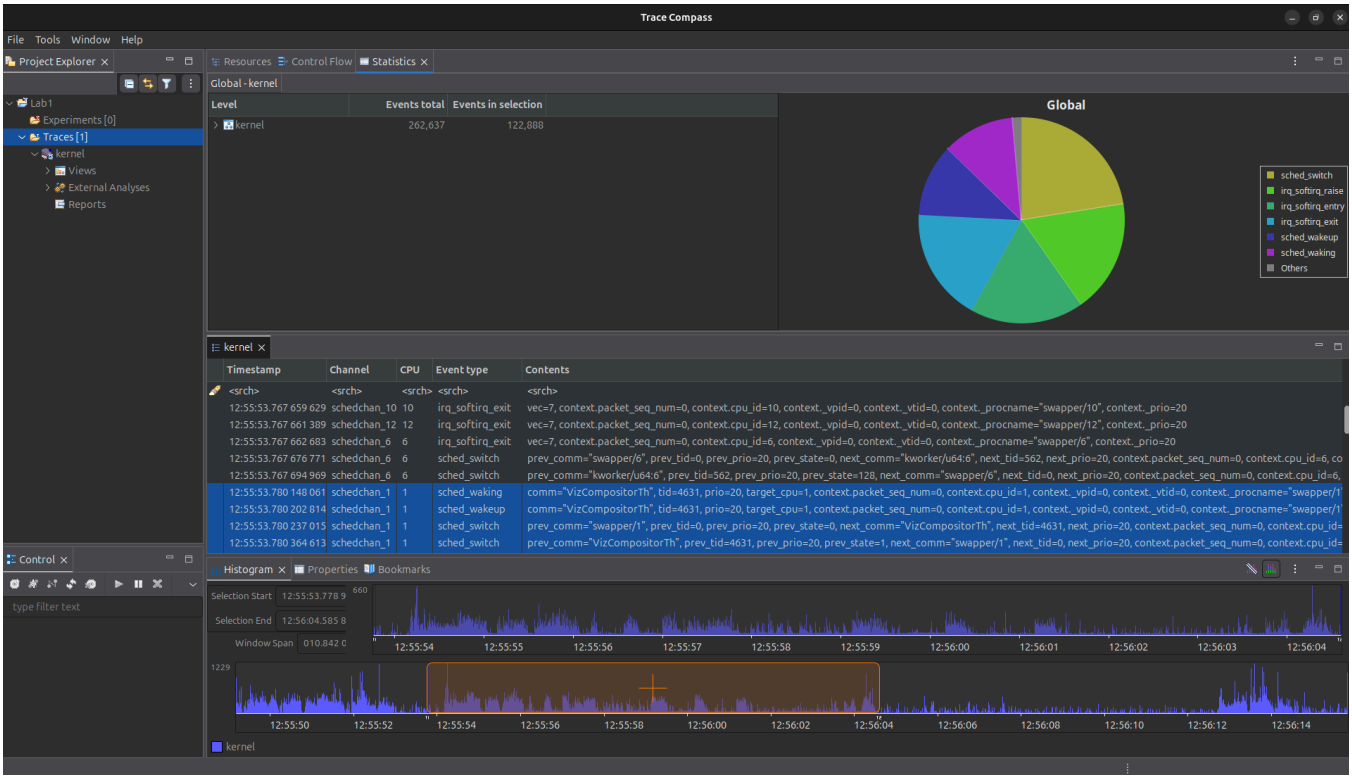


Figure 9 — Global Statistics. Trace Compass reports 262,637 events for the complete trace and 122,888 for the selected interval; the pie chart shows the global event-type distribution.

4.5. Control Flow: identifying the workers

In Trace Compass Control Flow, I identified the `stress-ng` parent (PID 8743) and four matrix workers (TIDs **8747**, **8748**, **8749**, and **8750**). I first inspected a short interval near their startup and then zoomed out to cover approximately **12:55:53.78–12:56:04.59**, which included the full matrix workload.

The four worker timelines showed sustained running activity during the workload. This observation was consistent with `stress-ng` reporting nearly 40 seconds of total user CPU time across a ten-second run. I used the colored segments to inspect scheduling states and the CPU Usage table to obtain execution-time measurements.

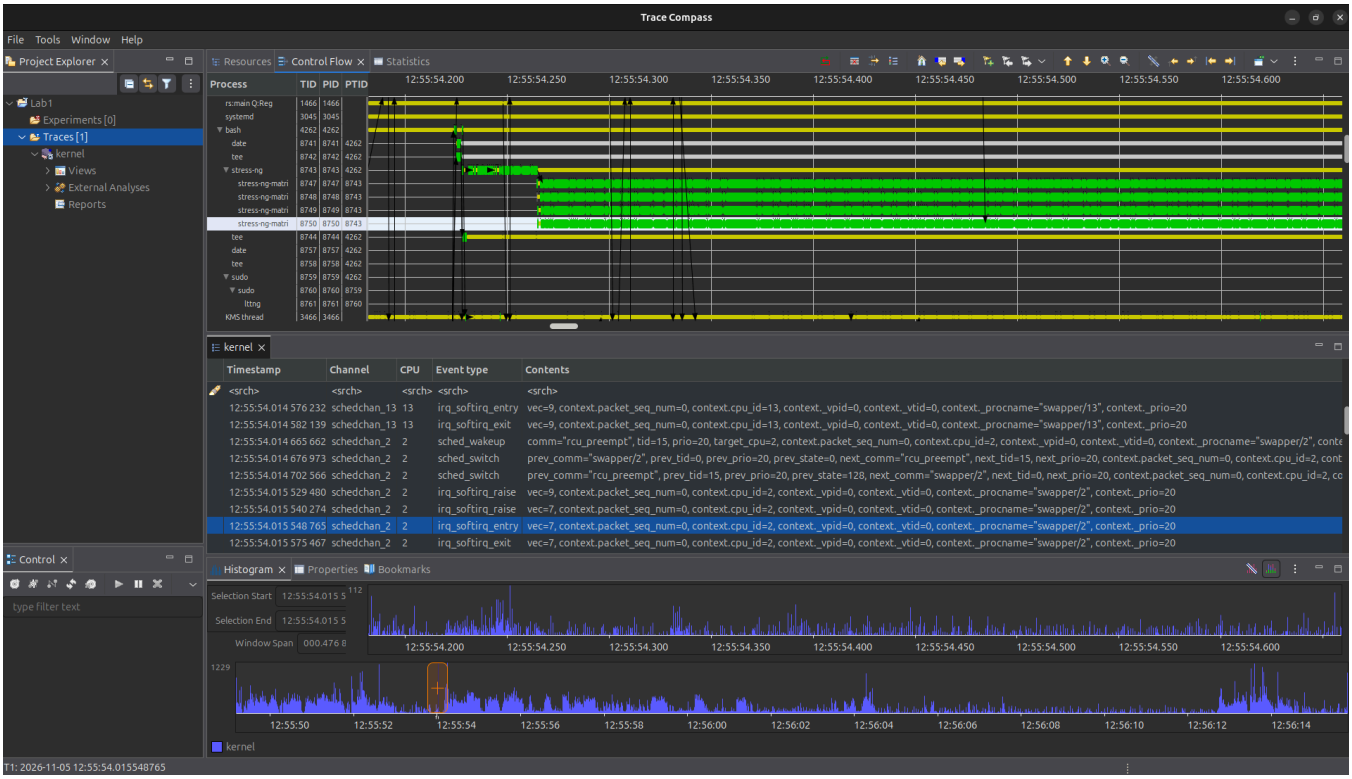


Figure 10 — Control Flow near worker startup. The expanded process tree identifies parent PID 8743 and worker TIDs 8747–8750, with their execution beginning near 12:55:54.265.

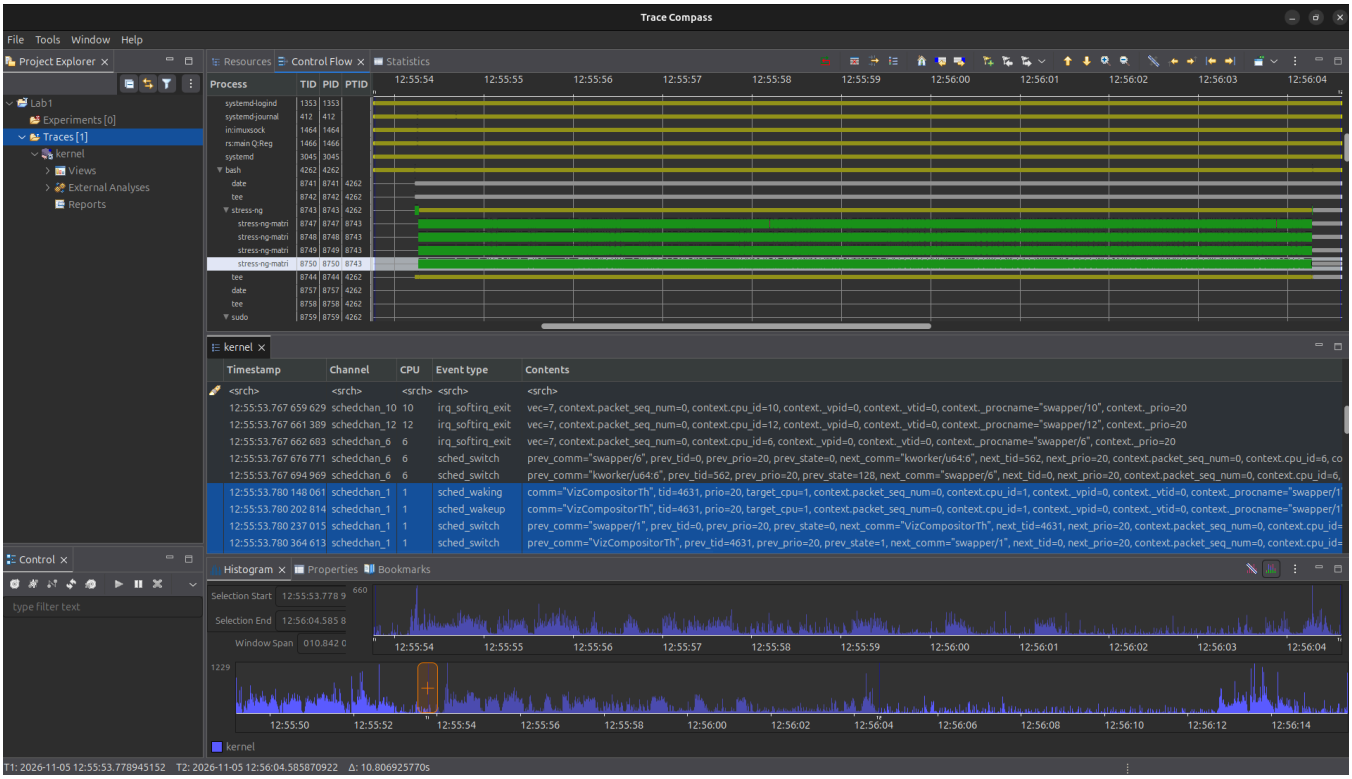


Figure 11 — Control Flow for the full workload. The selected interval spans approximately 12:55:53.78–12:56:04.59 and includes sustained activity for all four workers.

4.6. Resources and CPU usage

I used the Resources view to inspect the execution of individual threads on logical CPUs. During the main computational phase, the workers were primarily visible on CPUs **6, 10, 3, and 4** for TIDs **8747, 8748, 8749, and 8750**, respectively. I also observed IRQ and soft-IRQ activity in the Resources view.

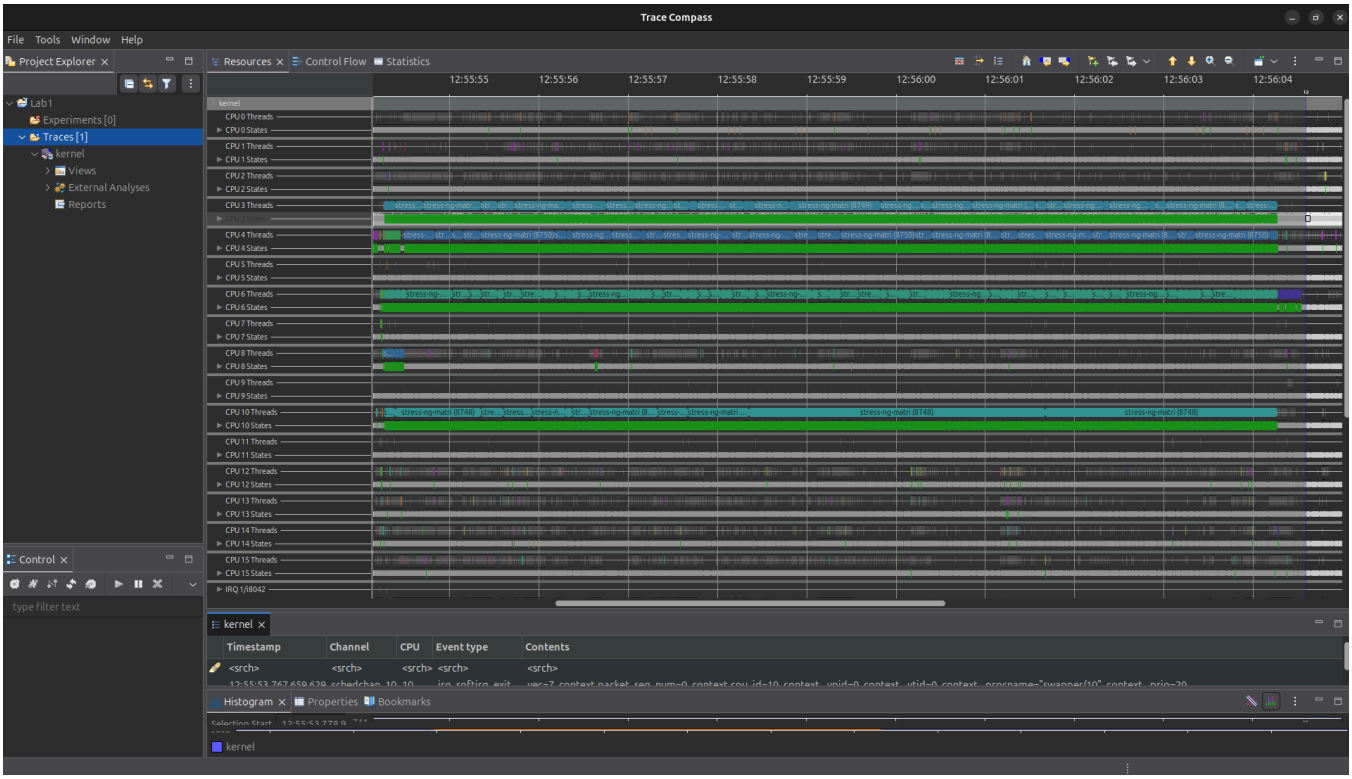


Figure 12 — Resources view: worker CPU placement. Long worker execution segments appear on CPUs 3, 4, 6, and 10 during the computational phase.

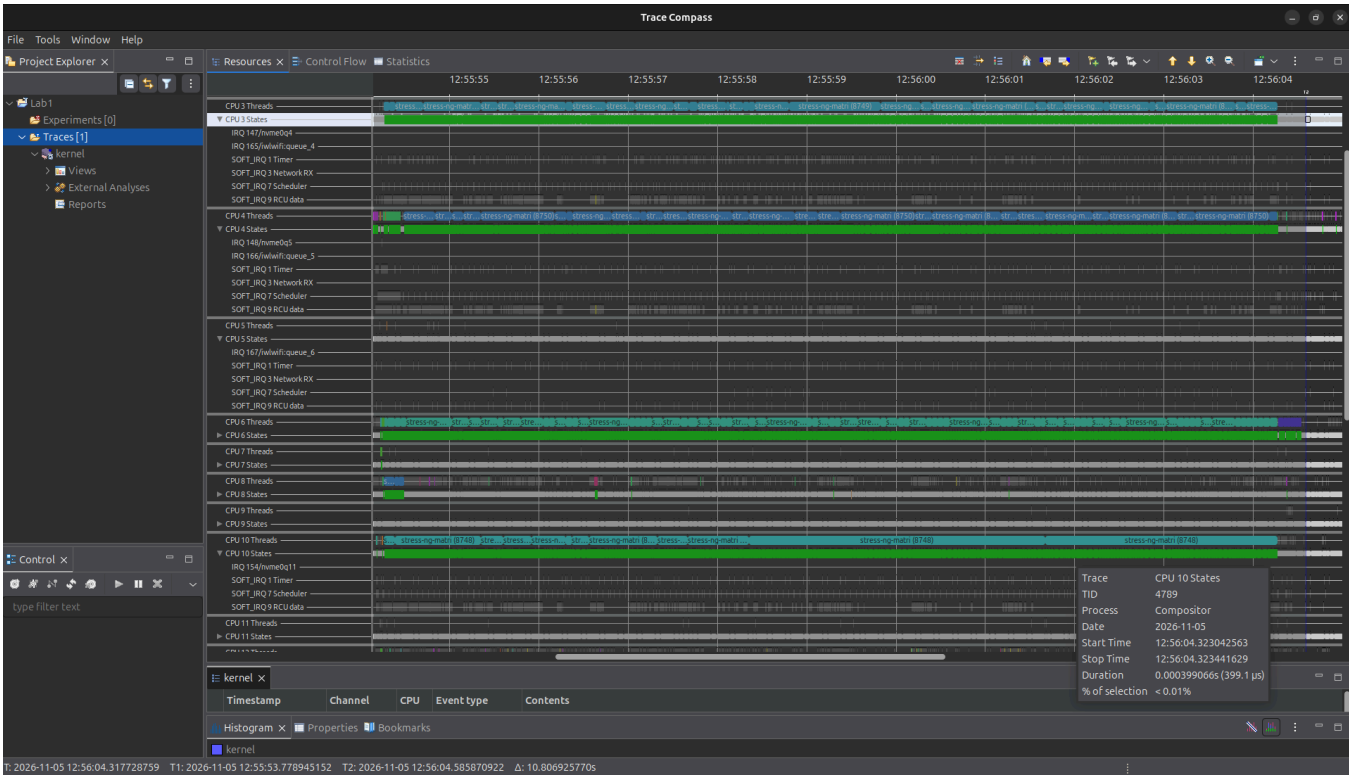


Figure 13 — Inspecting an individual CPU state interval. The tooltip identifies `Compositor` TID 4789 on CPU 10 at approximately 12:56:04.323, after the matrix workers have exited. This is background activity, separate from the worker execution segments.

The CPU Usage view showed total system usage near **400%** during the workload, consistent with approximately four logical CPUs being kept busy. I then inspected the per-worker table and obtained:

Worker TID	CPU execution time
8747	10.002 s
8748	10.001 s
8749	10.000 s
8750	10.001 s
Sum	**40.004 s**

I compared **40.004 s** of Trace Compass CPU time with the **39.99 s** cumulative user CPU time reported by `stress-ng`. Their close agreement supported the conclusion that all four workers executed CPU-intensive work in parallel for nearly the entire ten-second workload.

The per-worker curves overlapped near 100% when selected together, which was expected because their CPU usage patterns were similar. The roughly 59% values in the wider CPU Usage table were calculated over the selected analysis interval, which included time before and after the ten-second workload.

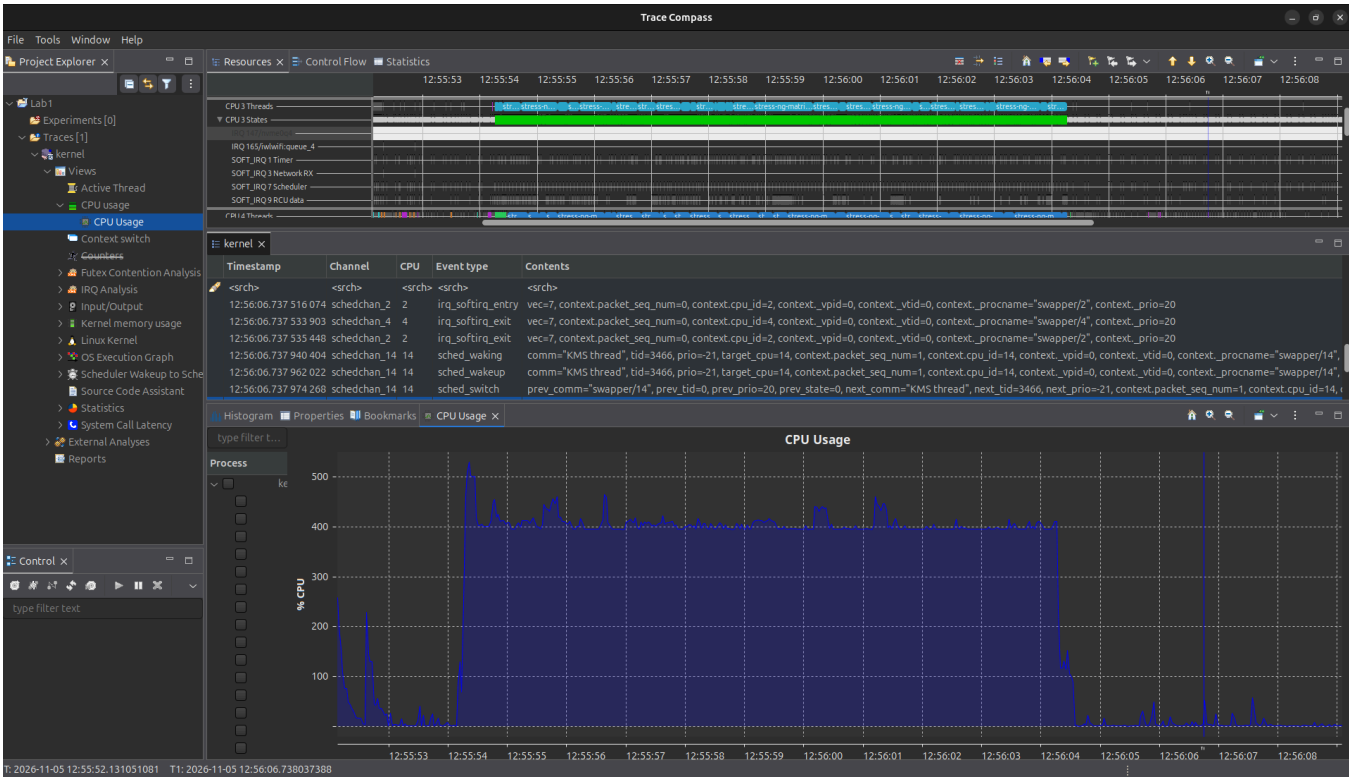


Figure 14 — CPU Usage across the workload boundaries. The overview shows the rise near 12:55:54, the sustained plateau, and the drop after 12:56:04.

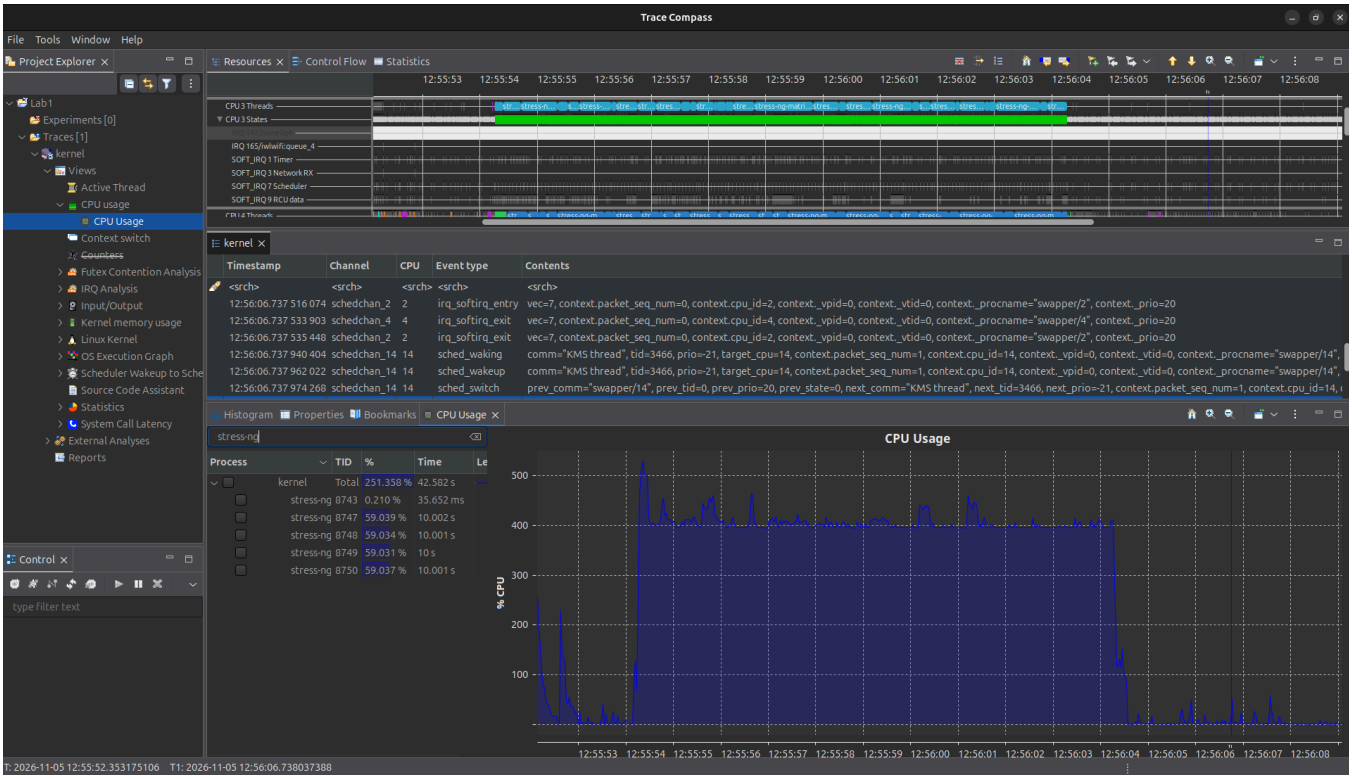


Figure 15 — CPU execution time for the four workers. The filtered table reports 10.002, 10.001, 10.000, and 10.001 seconds for TIDs 8747–8750. Its roughly 59% values refer to the wider displayed analysis interval.

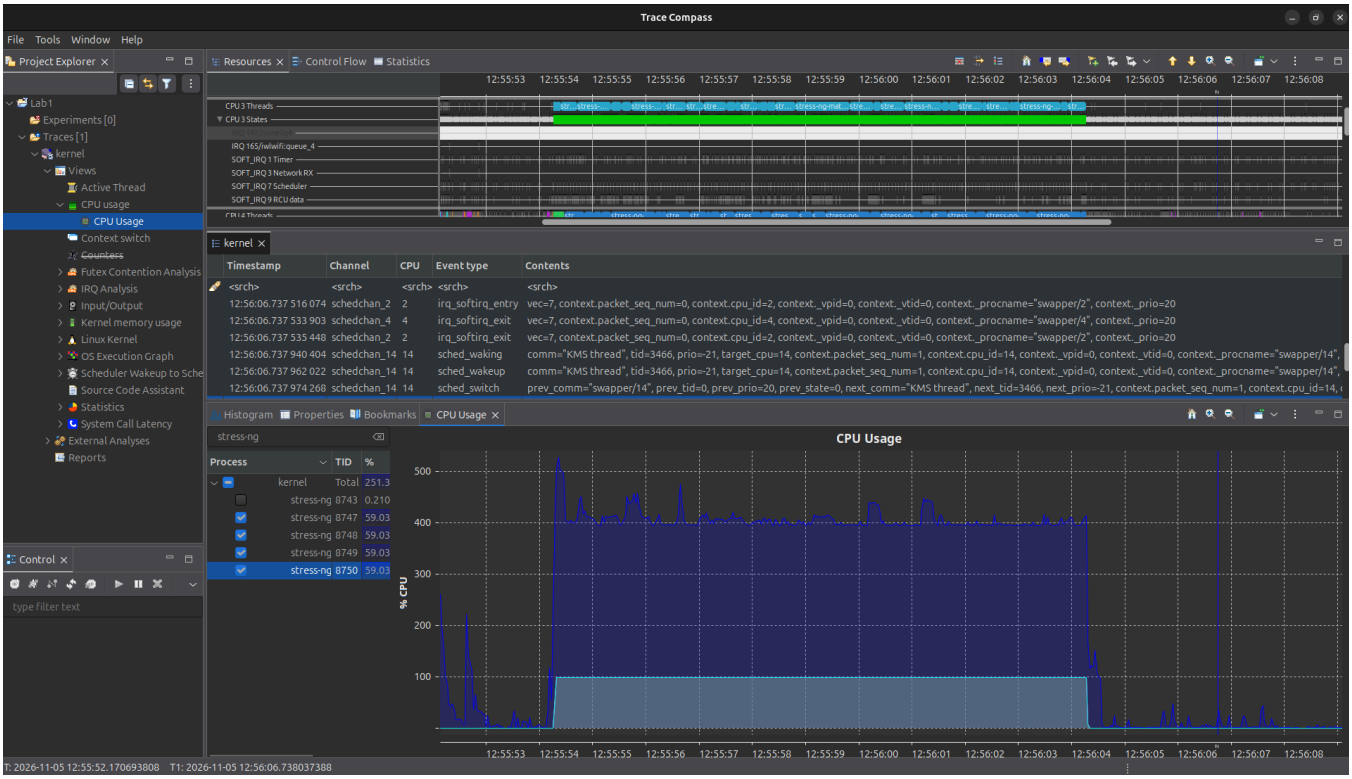


Figure 16 — Overlapping per-worker CPU usage curves. Selecting all four workers shows their similar curves near 100% during computation, alongside the aggregate system curve near 400%.

4.7. CPU placement changes

To analyze CPU placement more precisely, I parsed `sched_switch` records from the CTF data. For each worker, I counted `next_tid` appearances, extracted `cpu_id`, and counted a placement change whenever the CPU of a worker's next observed schedule-in differed from that of its preceding

schedule-in.

Worker TID	Schedule-ins	CPUs observed	Observed CPU changes	Schedule-ins by CPU
8747	85	6, 13	1	CPU 6: 82; CPU 13: 3
8748	14	10	0	CPU 10: 14
8749	36	3	0	CPU 3: 36
8750	34	4, 8, 13, 14	3	CPU 4: 28; CPU 8: 3; CPU 13: 2; CPU 14: 1
Total	**169**	—	**4**	—

The observed CPU sequences included **13 → 6** for TID 8747 and **13 → 14 → 8 → 4** for TID 8750. TIDs 8748 and 8749 were always scheduled onto CPUs 10 and 3, respectively, in the captured `sched_switch` data. I concluded that the workers settled into relatively stable CPU placements after brief startup movement.

I derived the **four observed CPU placement changes** by comparing CPU IDs between successive worker schedule-ins in the captured `sched_switch` records.

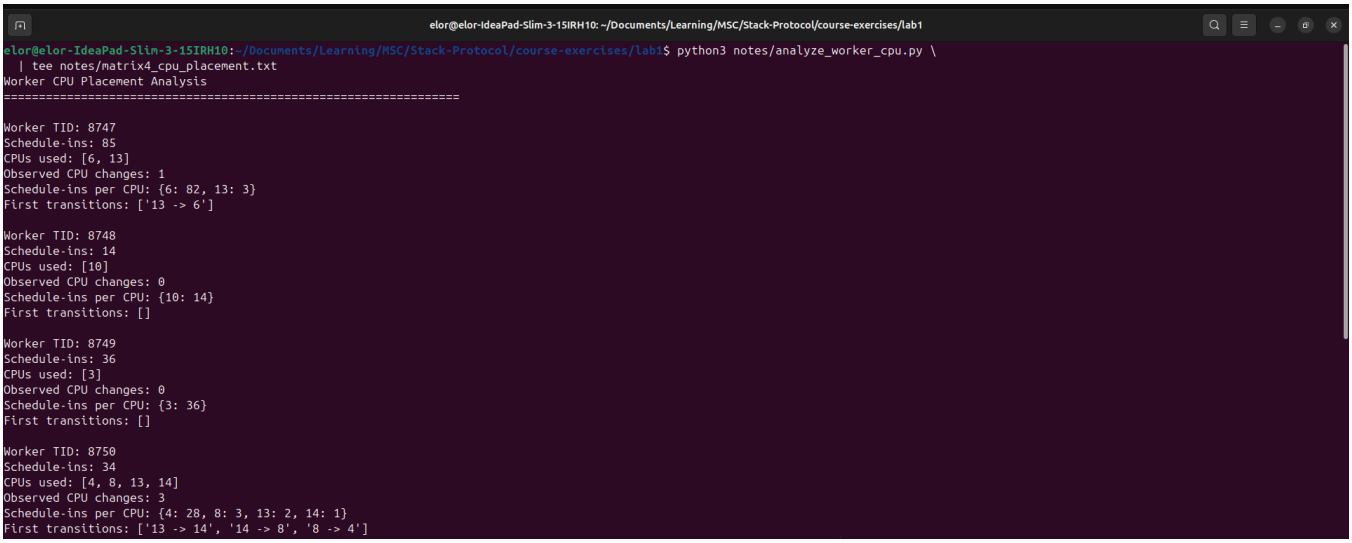


Figure 17 — CPU placement analysis from `sched_switch`. The script output records one observed CPU change for TID 8747, three for TID 8750, and none for TIDs 8748 and 8749.

4.8. Scheduler states and wakeups

I also parsed `sched_switch` records where `prev_tid` matched a matrix worker, and classified the recorded `prev_state` values. I used the following interpretations for the observed Linux task states:

- `0`: **TASK_RUNNING / runnable** — the task remained eligible for execution when switched out.
- `1`: **TASK_INTERRUPTIBLE** — the task entered an interruptible sleep.
- `32`: **EXIT_ZOMBIE** — the task was in a process-exit state.

Worker TID	Switch-ins	Switch-outs	Runnable (0)	Sleep (1)	Exit (32)
8747	85	85	82	2	1
8748	14	14	11	2	1
8749	36	36	33	2	1
8750	34	34	31	2	1
Total	**169**	**169**	**157**	**8**	**4**

Of the **169** worker switch-outs, **157 (92.9%)** had `prev_state = 0`. This proportion of switch-out events showed that the workers generally remained runnable, consistent with a CPU-bound workload. I used `prev_state` to classify each outgoing worker's scheduling state.

Each worker also had **two `sched_waking` events**, **two `sched_wakeup` events**, and **one `sched_wakeup_new` event**. All eight switch-outs with `prev_state = 1` occurred very near workload startup, approximately between **12:55:54.264807** and **12:55:54.265179**. The four exit-state switch-outs occurred near **12:56:04.267**, at the end of the workload. These observations supported the interpretation of short startup sleeps followed by sustained CPU-bound execution.



Figure 18 — Worker scheduler-state summary. The terminal output shows each worker's switch counts and `prev_state` distribution, together with wakeup-event counts.

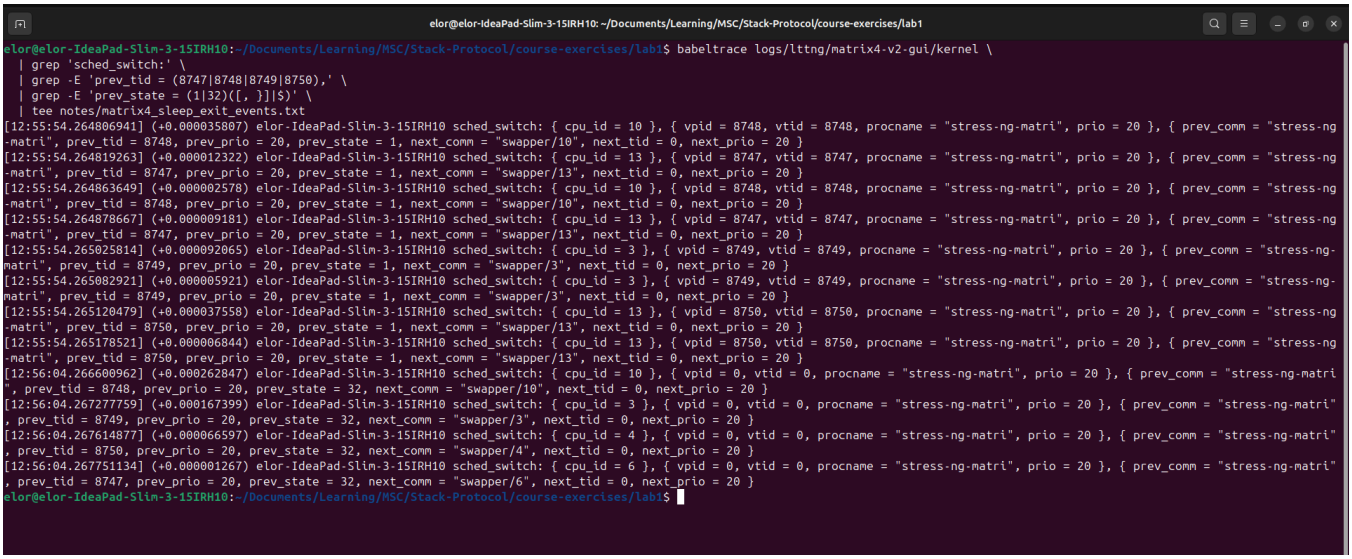


Figure 19 — Timing of startup sleeps and exit-state switch-outs. The filtered records show eight `prev_state = 1` events near 12:55:54.265 and four `prev_state = 32` events near 12:56:04.267.

Finally, I separately verified a `sched_process_exit` event for each worker:

Worker TID	`sched_process_exit` time	CPU	`group_dead`
8748	12:56:04.266338	10	1
8749	12:56:04.267110	3	1
8750	12:56:04.267404	4	1
8747	12:56:04.267548	6	1

The first and last worker-exit events were separated by about **1.21 ms**. The `group_dead = 1` values were consistent with each worker's thread group ending. This agreed with the `stress-ng` summary of four passed workers and zero failed workers.

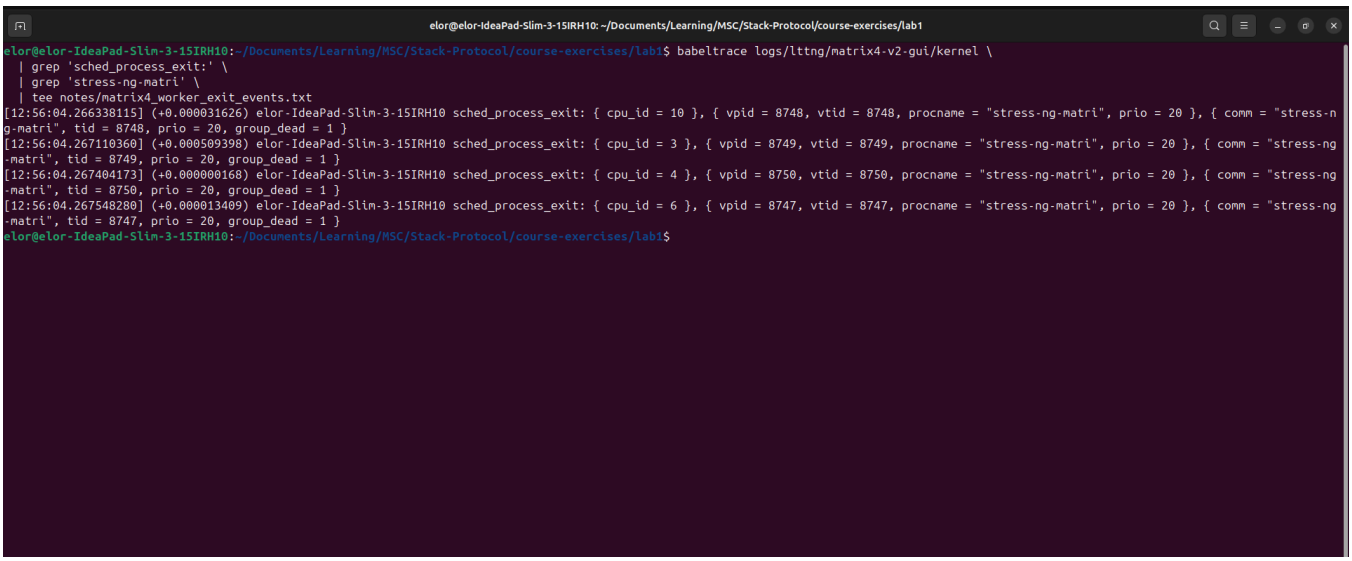


Figure 20 — Worker process-exit events. The filtered `sched_process_exit` records identify all four workers, their exit timestamps and CPUs, and `group_dead = 1` for each.

4.9. How I interpreted the scheduler’s behavior

I used the trace events to reconstruct the main scheduler steps. When a task was created, `sched_process_fork` identified its creation, while `sched_wakeup_new` showed its initial transition toward the runnable state. Later `sched_waking` and `sched_wakeup` events showed attempts to make sleeping tasks runnable. In `sched_switch`, the kernel identified the outgoing and incoming tasks on a particular CPU. The outgoing task’s `prev_state` helped me distinguish a task that remained runnable from one that blocked or exited. The paired CPU IDs and task IDs allowed me to follow execution placement across successive scheduling decisions.

The scheduler selects runnable tasks for execution on available CPUs; becoming runnable makes a task eligible for selection. In this experiment, `sched_switch` identified each switch from an outgoing task to an incoming task. A worker switched out with `prev_state = 0` remained runnable and could be selected again, while one switched out with `prev_state = 1` entered an interruptible sleep and needed to become runnable again before resuming execution. I observed two short startup sleeps per worker and a few initial CPU placement changes, followed by sustained computation primarily on CPUs 6, 10, 3, and 4 for workers 8747–8750, respectively.

The pattern of 157 runnable switch-outs out of 169, nearly equal cumulative CPU time, and subsequently stable CPU placements was consistent with the scheduler distributing four CPU-bound workers across available logical CPUs while also servicing other runnable tasks and interrupts. The 92.9% value summarizes the proportion of switch-out events where workers remained runnable, while the measured CPU execution times show how long they ran. I interpreted the recorded kernel `prio` field as a scheduling-priority value.